Dhaka International University

Lab Report details:

Lab Report Topic : **Write a program to solve Bankers algorithm using C**
Lab Report No : 06
Course Title : Database Management System
Course Code : 0613-301

Submitted By:	Submitted To:
S M Nabil Ausaf Roll: 01 Reg No : CS-D-91-23-124492	Md. Nazmus Sakib, Lecturer of CSE Department

Date of Submission:18.01.26

Lab No: 06

Lab Name

Implementation of Page Replacement Algorithms (FIFO, LRU, Optimal)

Objectives

- To understand the concept of page replacement in Operating Systems.
- To implement FIFO, LRU, and Optimal page replacement algorithms using C/C++.
- To compare the performance of different page replacement techniques in terms of page faults.

Background Study (with Example)

In an operating system, page replacement algorithms are used when a page fault occurs and memory is full. The OS must decide which page to remove from memory to make space for a new page.

FIFO (First In First Out)

The oldest page in memory is replaced first.
Example:
Reference string: 7 0 1 2 0 3 0 4
Frames: 3

LRU (Least Recently Used)

The page that has not been used for the longest time is replaced.

Optimal

The page that will not be used for the longest period in the future is replaced. This algorithm gives the minimum possible page faults.

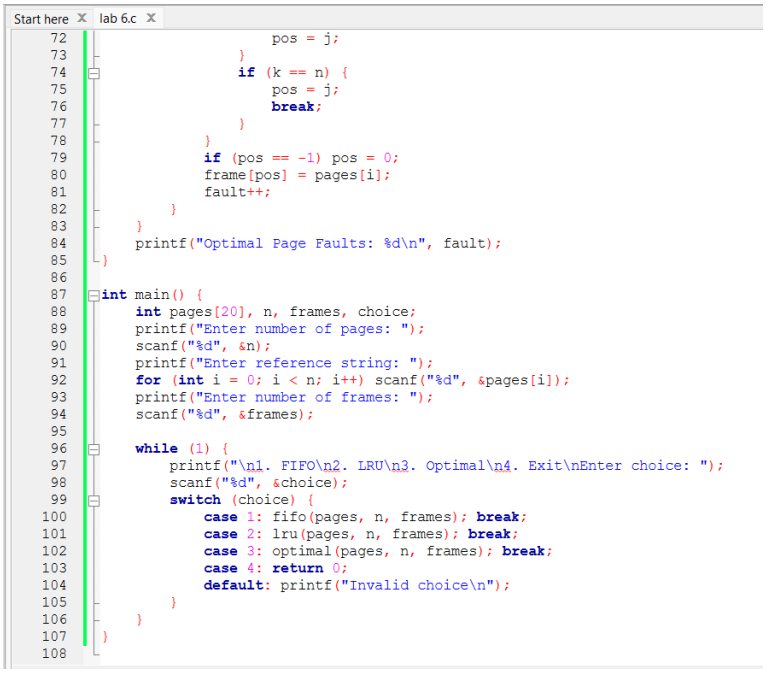
Code (with Results)

Start here | lab 6.c

```c
#include <stdio.h>

void fifo(int pages[], int n, int frames) {
    int frame[10], fault = 0, index = 0;
    for (int i = 0; i < frames; i++) frame[i] = -1;

    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
            frame[index] = pages[i];
            index = (index + 1) % frames;
            fault++;
        }
    }
    printf("FIFO Page Faults: %d\n", fault);
}

void lru(int pages[], int n, int frames) {
    int frame[10], time[10], fault = 0, counter = 0;
    for (int i = 0; i < frames; i++) frame[i] = -1;

    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }
```

Start here | lab 6.c

```c
        if (!found) {
            int min = 0;
            for (int j = 1; j < frames; j++)
                if (time[j] < time[min]) min = j;
            counter++;
            frame[min] = pages[i];
            time[min] = counter;
            fault++;
        }
    }
    printf("LRU Page Faults: %d\n", fault);
}

void optimal(int pages[], int n, int frames) {
    int frame[10], fault = 0;
    for (int i = 0; i < frames; i++) frame[i] = -1;

    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
            int pos = -1, farthest = i + 1;
            for (int j = 0; j < frames; j++) {
                int k;
                for (k = i + 1; k < n; k++) {
                    if (frame[j] == pages[k]) break;
                }
                if (k > farthest) {
                    farthest = k;
                    pos = j;
                }
```



Sample Result

For reference string: 7 0 1 2 0 3 0 4
Frames: 3

- FIFO Page Faults: 6
- LRU Page Faults: 5
- Optimal Page Faults: 4

Discussion

FIFO is simple but inefficient as it may replace frequently used pages. LRU performs better by considering recent usage but requires more overhead. Optimal gives the best result but is not practically implementable as it needs future knowledge. Hence, LRU is commonly used in real systems.